OutOfRole: Getting the Data

The Debugging Table, IBM Z Datathon 2026

October 17, 2026

What we use

The **CERT Insider Threat Test Dataset**, published by Carnegie Mellon's Software Engineering Institute: synthetic activity logs from January 2010 to May 2011. We use two releases: **r4.2** (1,000 employees, **70 labeled malicious insiders**, three scenarios), where every setting was chosen, and **r5.2** (2,000 employees, **99 insiders**, four scenarios), where the frozen settings were tested. We also need the **answer key**, which names the insiders and their exact malicious events (used only for grading).

- Dataset page: https://kilthub.cmu.edu/articles/dataset/Insider_Threat_Test_Dataset/12841247
- Citation DOI: 10.1184/R1/12841247

Download **r4.2**, **r5.2** and **answers**. The other releases are not needed. r4.2 alone is enough for `build_tables.py` and `outofrole_minimal.py` on one release; the notebook and some scripts in `code/experiments/` expect both.

Files and direct links

File	Size	Direct download	MD5
r4.2.tar.bz2	4.82 GB	https://ndownloader.figshare.com/files/24856766	cf64caa378acb77cd0c608a5576d998c
r5.2.tar.bz2	11.14 GB	https://ndownloader.figshare.com/files/24849938	e24c10807f4923b731276c3421a13c9f
answers.tar.bz2	1.25 MB	https://ndownloader.figshare.com/files/24857828	c0c96eaa06915e417504f932756f58d8
README (SEI)	1.4 KB	https://ndownloader.figshare.com/files/24860765	3168963335887132baccc4179f46c467

Links and checksums come from the figshare API for article 12841247.

Download into the repo's data/ folder

Every script looks for the data in the `data/` folder at the top of the repo. It finds the repo from its own file location, so it runs from any working folder, but there is no setting that points it somewhere else. Download straight onto the machine that will run the code (on the LinuxONE VM, so the archives never have to be uploaded from a laptop). After cloning the repo:

```
cd IBMZDatathon2026                    # the repo root
mkdir -p data/raw && cd data/raw
curl -L -C - -o r4.2.tar.bz2    https://ndownloader.figshare.com/files/24856766
curl -L -C - -o r5.2.tar.bz2    https://ndownloader.figshare.com/files/24849938
curl -L -C - -o answers.tar.bz2 https://ndownloader.figshare.com/files/24857828
md5sum r4.2.tar.bz2 r5.2.tar.bz2 answers.tar.bz2      # compare with the table above
mkdir -p ../r4.2 ../r5.2 ../answers
```

```
tar -xjf r4.2.tar.bz2    -C ../r4.2
tar -xjf r5.2.tar.bz2    -C ../r5.2
tar -xjf answers.tar.bz2 -C ../answers
```

Each archive holds a top folder of the same name, so this gives exactly the layout the code reads:

Folder	What is in it	Read by
`data/r4.2/r4.2/`, `data/r5.2/r5.2/`	Raw logs and `LDAP/`	`build_tables.py` (logon, device, file, http); `LDAP/` by `followup_checks.py`, `least_privilege_full.py` and the notebook
`data/answers/answers/`	Answer key	Grading only: `outofrole_minimal.py`, the experiments and the notebook
`data/cache/r4.2/`, `data/cache/r5.2/`	The three Spark tables: `daily`, `pc_pairs`, `dom_pairs`	Written by `build_tables.py` (or the notebook's Step 1); read by everything else

Keep the doubled folder names (`r4.2/r4.2`, `answers/answers`): the paths are fixed in the code, and a flat `data/r4.2/logon.csv` is not found. **Data on another disk:** from the repo root, make `data` a symbolic link to it (`ln -s /bigdisk/outofrole-data data`); Spark and pandas both follow the link. Git does not apply the `data/` rule in `.gitignore` to a link, so also run `echo data >> .git/info/exclude` once. Leave `OUTOFROLE_CACHE` unset: `build_tables.py` would write the tables there, but the other scripts read only `data/cache/`. A real `data/` folder is git-ignored, so nothing in it can be committed by accident.

`-C -` resumes an interrupted download, so rerun the same line if the connection drops. On a Mac, use `md5 -q` instead of `md5sum`. At 7 MB/s expect about 12 minutes for r4.2 and 27 for r5.2. Each archive plus its extracted files needs free disk of about **20 GB for r4.2 and 50 GB for r5.2** (`df -h .` to check); the Spark tables add under 0.1 GB. The code reads only `logon.csv`, `device.csv`, `file.csv`, `http.csv` and `LDAP/`, so on a short disk extract just those (this skips `email.csv`, 1.4 GB in r4.2 and 9.4 GB in r5.2): `tar -xjf r4.2.tar.bz2 -C ../r4.2 --wildcards "*logon.csv" "*device.csv" "*file.csv" "*http.csv" "*LDAP*"`, and the same for r5.2. Delete `data/raw/*.tar.bz2` afterwards if space is tight.

What is inside

Dataset (`data/r4.2/r4.2/` after extracting; 16 GB in total):

File	Size	Rows	Columns	Use
logon.csv	58.5 MB	854,859	id, date, user, pc, activity	Core: who logs into which PC, and when
device.csv	29.0 MB	405,380	id, date, user, pc, activity	Core: USB connect and disconnect
file.csv	193 MB	445,581	id, date, user, pc, filename, content	Core: files copied to USB
LDAP/ (18 files)	0.1 MB each	1 file per month, Dec 2009 to May 2011	employee_name, user_id, email, role, business_unit, functional_unit, department, team, supervisor	Core: role and department for peer groups
psychometric.csv	45 KB	1,000	employee_name, user_id, O, C, E, A, N	Skip: personality scores

File	Size	Rows	Columns	Use
email.csv	1.36 GB		id, date, user, pc, to, cc, bcc, from, size, attachments, content	Not used
http.csv	14.5 GB		id, date, user, pc, url, content	Core: web visits (daily counts and the website recommender); Spark reads it once
readme.txt, license.txt	small			Read license.txt once (see below)

r5.2 (`data/r5.2/r5.2/`, 40 GB) has the same files for 2,000 people (http.csv 30.1 GB, email.csv 9.4 GB), plus `decoy_file.csv`, which is not used. Its `file.csv` adds `activity`, `to_removable_media` and `from_removable_media`; `build_tables.py` keeps only the rows with `to_removable_media` True, while r4.2 has no such column and every row counts.

Answer key (`data/answers/answers/`):

- `insiders.csv`: one row per insider: dataset, scenario, details file, user ID, start and end time. Filter to `dataset == 4.2` for our 70 rows (`5.2` for 99).
- `r4.2-1/`, `r4.2-2/`, `r4.2-3/`: one file per insider (30, 30 and 10 files) listing that insider's malicious events; `r5.2-1/` to `r5.2-4/` likewise (29, 30, 10 and 30 files). These are **not proper CSV**: rows vary in length and the first column says which log the row came from.
- `scenarios.txt`: plain-English descriptions. Scenario 1 is after-hours USB use plus uploads to wikileaks.org; scenario 2 is job hunting followed by heavy thumb-drive data theft; scenario 3 is a disgruntled system administrator with a keylogger; scenario 4 (r5.2 only) is a user who logs into other people's computers, searches for files and emails them home, more and more often over three months.

Rules and housekeeping

- **Downloading data before the event is allowed.**
- **Never commit the data to GitHub.** `data/` is already in the repo's `.gitignore`. The final submission only needs the code link.
- Credit the dataset in the README and video, with the page link and DOI above.
- **Do not pass copies around.** Each teammate or VM downloads from the CMU links above.
- If we show actual rows in the video or README, add the notice "Copyright 2011 ExactData, LLC, All Rights Reserved." Summaries and charts of the data need no notice.

Then run it

```
pip install -r requirements.txt        # Python 3.11, Java 17; LinuxONE notes inside
python code/build_tables.py 4.2        # Spark: raw logs to data/cache/r4.2 (once)
python code/outofrole_minimal.py 4.2   # every headline number, about 20 seconds
python code/case_file.py               # the Granite step's self-test, needs no data
```

Repeat the first two `python` lines with `5.2` for the test release. The scripts run from any folder. The notebook `evidence/Study_Notebook.ipynb` expects `evidence/` as its working folder (Jupyter's default), because its last cells call `../code/`.